SN2N for Cross-cumulant SOFI data Code manual

Dorus van den Broek
Leiden University
Netherlands
dvdbroek@strw.leidenuniv.nl

Casper Bonte
TU Delft
Netherlands
C.bonte@student.tudelft.nl

Jelle Wagenaar
TU Delft
Netherlands
J.J.Wagenaar@student.tudelft.nl

Niek Paridaens
TU Delft
Netherlands
N.T.Paridaens@student.tudelft.nl

Xinyue Wang
TU Delft
Netherlands
X.Wang-89@student.tudelft.nl

Tijmen van den Kieboom
TU Delft
Netherlands
T.J.vandenkieboom@student.tudelft.nl

Contents

1 Code introduction

This workflow implements a Self-Supervised pipeline to denoise Super-resolution Optical Fluctuation Imaging (SOFI) data. Unlike traditional supervised learning that requires clear ground truth images, this project uses Self-Supervised Noise2Noise (SN2N) strategy adapted for single images.

Diagonal resampling is used to convert the images into a image paired. After this Fourier up scaling is applied to get the original resolution back. After this optional RL-deconvolution is used to deconvolution the PSF.

One of the core assumptions we are making is that the noise is independent pixel-to-pixel.

2 Setup

2.1 Software and libraries

We recommend creating a fresh (conda) environment to avoid conflicts. Make sure to use python 3.8+ preferably python 3.9. Then install pytorch with your version of CUDA or ROCm (for ROCm, see the ROCm docker image, NOTE: This is not tested). Than install the following libraries using pip.

- Numpy: 2.2.6
- tifffile: 2025.12.20
- matplotlib: 3.10.3
- tqdm: 4.67.1
- pandas: 2.3.3
- skimage: 0.26.0
- scipy: 1.15.3

2.2 Create Data Directories

The trainer expects (or creates) the following parent directories:

- **dataset/** - Generated training patches (pairs)
- **models/** - Saved models
- **images/** - Preview of predictions during training

3 File structure

File name	Primary responsibility	Key public symbols
SN2N_resampling_fourier_RL _deconvolution _augmentation_py.py	Preprocessing pipeline and sample generation.	run_pipeline; make_sn2n_sample; build_sn2n_sample
improved_trainer.py	2D training loop, loss definitions, checkpointing.	net2D; gradient_loss; EarlyStopping
models.py	Neural network architectures (2D U-Net, 3D U-Net scaffolding).	Unet_2d; Unet_3d
eval_utils.py	Dataset mapping and evaluation on test/validation splits, hyperparameter search.	load_and_standardize_mapping; process_test_set_with_model; process_val2_with_model
metrics_utils.py	SQUIRREL-style metrics and SSIM helpers.	squirrel_rsp_rse; squirrel_ssim; fit_affine_percentiles
plotting_utils.py	Simple plotting utilities.	plot_loss_curve
get_options.py	ArgumentParser factories for (hypothetical) CLI entrypoints.	datagen2D/3D; trainer2D/3D; Predict2D/3D; execute2D/3D

3.1 Preprocessing File

The Preprocessing pipeline is split up into 4 parts: diagonal-resampling, Fourier up scaling, RL deconvolution and data augmentation. This is done by the file SN2N_resampling_fourier_RL _deconvolution _augmentation_py.py. With the following public API's:

- **block2d(img2d):** even-crops input and returns two diagonal-sampled views (left, right).
- **fourier_zoom(img2d, scale=2):** Fourier-domain zero padding to upsample by an integer scale factor.
- **rl_deconvolve(image, psf, iters=10, floor=1e-3):** Richardson–Lucy deconvolution with normalization and floor for stability.
- **random_interchange(mode, patch_size, imga, imgb=None):** patch-interchange augmentation (single-image or cross-image).
- **sliding_window_2d(img, patch_size, stride, mode, filter_val):** extract candidate patches and reject low-intensity patches.
- **basic_augment(img, mode):** deterministic flip/rotation augmentation.
- **run_pipeline(tiff_path, ...):** end-to-end preprocessing returning all intermediate arrays in a dictionary.
- **make_sn2n_sample(left_rl, right_rl, p_low, p_high):** shared-percentile normalization and concatenation into an (H, 2W) float32 sample.
- **save_sn2n_sample(sample, out_path):** writes sample to TIFF.
- **build_sn2n_sample(tiff_path, save_path=None, ...):** convenience wrapper combining run_pipeline and make_sn2n_sample (with optional save).

3.2 Training File

The training functions are defined in improved_trainer.py. This file contains the following public API's:

- **gradient_loss(pred, target, proxy, tau_thr, softness=0.1, blur_kernel_size=3):** Sobel-gradient L1 loss under a soft mask.
- **EarlyStopping(patience):** tracks best metric (lower is better) and signals stop after N non-improving steps.
- **net2D(...):** trainer wrapper around Unet_2d, optimizer, scheduler, and checkpointing.
- **net2D.load_batch2d(...):** yields batches built from concatenated TIFF training samples.
- **net2D.train():** main training loop (AMP optional) with optional ReduceLROnPlateau and EarlyStopping.
- **net2D.validate():** validation loss on val_path patches.
- **net2D.test(test_img_np):** forward-pass a single 2D image (NumPy) through the trained model.

3.3 Evaluation Files

The functions needed for evaluation are defined in `eval_utils.py`. This file contains the following public API's:

- **load_and_standardize_mapping(csv_path, ...):** standardize expected columns (`img_path`, `img_file`, `gt_path`, `gt_exists`).
- **process_test_set_with_model(df_map_std, sn2n_model, sigma=2.0, verbose=True):** evaluate on a test split.
- **process_val2_with_model(df_val2_std, sn2n_model, verbose=True):** evaluate on a VAL2 split.
- **summarize_metrics(df_results):** compute means over successful rows.
- **score_from_metrics(m, w_rsp=2.0, w_rse=2.0, w_ssim=1.0):** compute a scalar score.
- **hyperparameter_search_train_val1_eval_val2(...):** train, validate, and evaluate across a hyperparameter grid.

This file uses the following API's from the file `metrics_utils.py`.

- **fit_affine_percentiles(x, p_low=1.0, p_high=99.5):** compute robust low/high percentiles.
- **apply_affine(x, lo, hi):** normalize $(x - lo)/(hi - lo)$ and clip to $[0, 1]$.
- **fit_alpha_beta(x, y):** least-squares affine fit $y \approx \alpha x + \beta$.
- **squirrel_rsp_rse(sr, gt, intensity_fit=True):** compute Pearson correlation and RMSE.
- **squirrel_ssim(sr, gt, intensity_fit=True, clip_to_gt=True):** compute SSIM.
- **add_metrics_labels_ssim(ax, rsp, rse, ssim_val=None):** annotate axes with metric values.

4 Reproducing the results

To reproduce the results you should run the following files in order. For more information per notebook you can check the first markdown cell.

Run Order:

- `Creating-preprocessed-folders-p2p_cleaned.ipynb`
- `Training-and-prediction_cleaned_refactored.ipynb`
- `Results-Visualizations.ipynb`

5 Custom Loss Function

For this workflow we have To enable the self-supervised learning, we designed a custom loss function. The total loss is a weighted sum of a self-supervised loss (LSN2N) and a masked gradient penalty to counteract the loss of high-frequency detail. The loss function is constructed with the following formula's:

$$
\begin{aligned}
L_{\text{total}} &= L_{\text{SN2N}} + \lambda_{\text{grad}} L_{\text{grad}} \\
L_{\text{grad}} &= \left\| \nabla f(x_{\text{input}}) - \nabla f(x_{\text{label}})_{\text{detached}} \right\|_1 \cdot M_{\text{proxy}} \\
L_{\text{SN2N}} &= \frac{L_1 + L_2 + \lambda_{\text{cons}} \cdot L_3}{2 + \lambda_{\text{cons}}} \\
L_1 &= \left\| f(x_{\text{input}}) - x_{\text{label}} \right\|_1 \\
L_2 &= \left\| f(x_{\text{label}}) - x_{\text{input}} \right\|_1 \\
L_3 &= \left\| f(x_{\text{label}}) - f(x_{\text{input}}) \right\|_1
\end{aligned}
\tag{1}
$$

6 Appendix

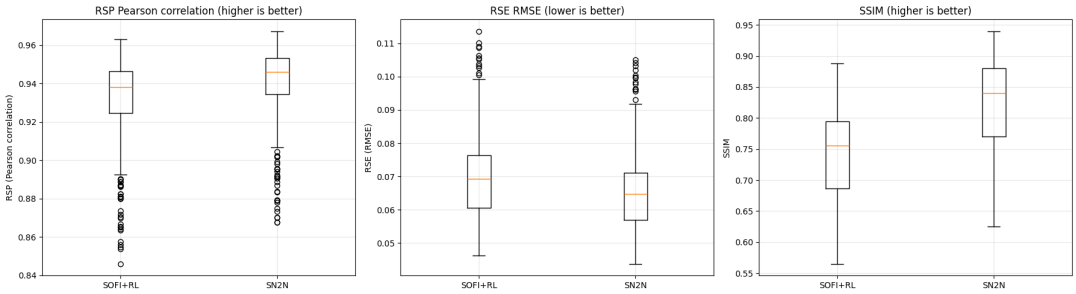

Figure 1: Image showing the metrics used to determine the quality of the denoised SOFI image

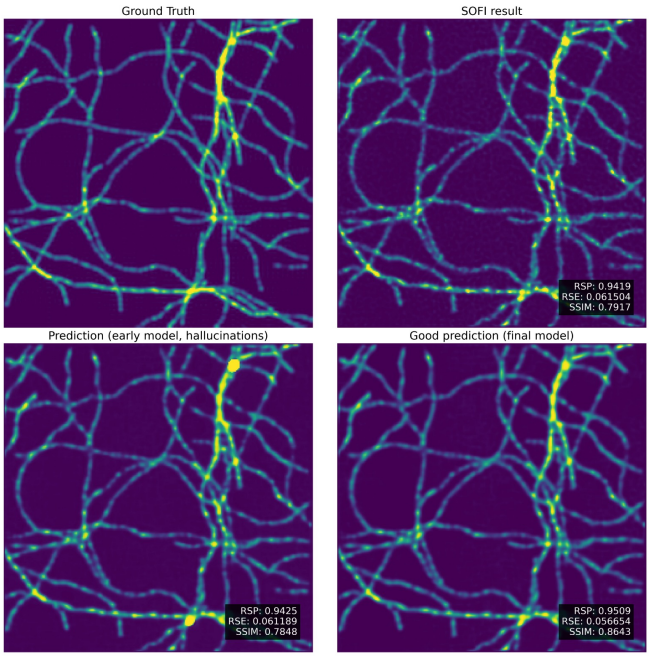

Figure 2: Image showing the inference of a SOFI image in respect to its ground truth

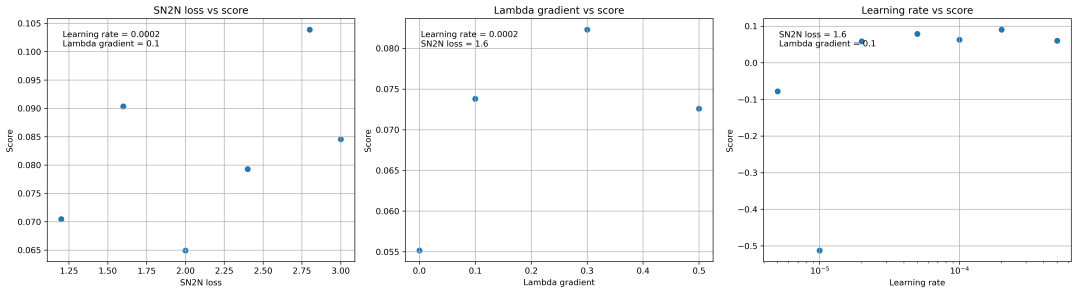

Figure 3: Image showing the score of the model during the hyper parameter fitting